

**INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
RAICHUR, KARNATAKA, 584135**



Modeling Physical Systems using Hamiltonian Neural Networks
Minor Project Report
VI Semester

Submitted By:

Amit Kumar CS23B1004
Prashant Kumar CS23B1058

Under the Guidance of

Produyti Pradhan

Department of Computer Science and Engineering

Session 2025-26

**INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
RAICHUR, KARNATAKA, 584135**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project entitled **Modeling Physical Systems using Hamiltonian Neural Networks** submitted by:

Amit Kumar Cs23b1004
Prashant Kumar Cs23b1058

is the partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering is an authentic work carried out by them under my supervision and guidance.

Dr. Priodyuti Pradhan

DECLARATION

We, hereby declare that the following report which is being presented in the Minor Project entitled as Modeling Physical Systems using Hamiltonian Neural Networks is an authentic documentation of our own original work to the best of our knowledge. The following project and its report in part or whole, has not been presented or submitted by us for any purpose in any other institute or organization. Any contribution made to the research by others, with whom we have worked at Indian Institute of Information Technology Raichur or elsewhere, is explicitly acknowledged in the report.

AMIT KUMAR CS23B1004 _____

PRASHANT KUMAR CS23B1058 _____

ACKNOWLEDGEMENT

With due respect, we express our deep sense of gratitude to our respected guide and coordinator Dr. Priodyuti Pradhan, for his valuable help and guidance. We are thankful for the encouragement that he has given us in completing this project successfully.

It is imperative for us to mention the fact that the report of minor project could not have been accomplished without the periodic suggestions and advice of our project guide Dr. Priodyuti Pradhan and project coordinators.

We are also thankful to all the other faculty, staff members and laboratory attendants of our department for their kind cooperation and help. Last but certainly not the least; we would like to express our deep appreciation towards our family members and batch mates for providing support and encouragement.

ABSTRACT

The accurate long-term simulation of conservative physical systems presents a fundamental challenge for purely data-driven machine learning models, which lack inherent mechanisms to enforce physical conservation laws. This report investigates physics-informed machine learning as a principled solution to this problem, using two canonical mechanical systems as benchmarks: the frictionless spring-mass system and the nonlinear pendulum. Two modelling paradigms are compared: a Standard Neural Network (NN) trained via supervised regression on state-transition pairs, and a Hamiltonian Neural Network (HNN) that learns a scalar Hamiltonian function from which system dynamics are derived through automatic differentiation.

The standard NN, despite achieving low one-step mean squared error during training, is demonstrated to violate the law of energy conservation severely during autoregressive trajectory rollout. Compounding prediction errors cause the simulated phase-space orbit to drift progressively, rendering the model physically inadmissible for long-horizon prediction tasks. In contrast, the HNN exploits the symplectic structure of Hamilton’s canonical equations,

$$\dot{q} = \frac{\partial \mathcal{H}}{\partial p}, \quad \dot{p} = -\frac{\partial \mathcal{H}}{\partial q}, \quad (0.1)$$

to guarantee that the learned dynamics respect the conservation of total mechanical energy. Qualitative evaluation via phase-space portraits and quantitative evaluation via total energy deviation confirm that HNN trajectories remain stable and physically consistent across the full simulation horizon on both systems.

The results establish that embedding structural physical priors into a neural network architecture through the Hamiltonian formalism yields superior generalisation, physical fidelity, and long-term stability compared to unconstrained black-box regression. This work underscores the necessity of physics-informed inductive biases in scientific machine learning applications.

Code Availability: The complete implementation, datasets, and experimental scripts used in this work are publicly available at: <https://github.com/Amitkr1311/Modeling-Physical-Systems-using-Hamiltonian-Neural-Networks>

TABLE OF CONTENTS

DECLARATION	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
LIST OF FIGURES	vi
LIST OF TABLES	vii
1 Introduction	viii
1.1 Background	viii
1.2 Motivation	viii
1.3 Objectives	ix
1.4 Systems Under Study	ix
1.4.1 Spring-Mass System	ix
1.4.2 Nonlinear Pendulum	x
1.5 Report Organisation	x
2 Literature Review	xi
2.1 Hamiltonian Neural Networks (Greydanus et al., 2019)	xi
2.1.1 What is the Work?	xi
2.1.2 How is it Done? Methods, Techniques, and Innovations	xi
2.1.3 Importance and Applications	xii
2.1.4 Drawbacks and Limitations	xii
2.1.5 Criticism and Link to Motivation	xiii
3 Gaps Identified in the Baseline Approach	xv
3.1 Absence of Physical Constraints in Standard Neural Networks	xv
3.2 Trajectory Divergence Under Autoregressive Rollout	xv
3.3 Violation of Energy Conservation	xvi
3.4 Poor Generalisation to Unseen Initial Conditions	xvi
3.5 Instability in Long-Horizon Prediction	xvi
4 Methodology	xviii
4.1 Data Generation	xviii
4.1.1 Spring-Mass Dataset	xviii
4.1.2 Pendulum Dataset	xix
4.2 Baseline Standard Neural Network	xix
4.2.1 Problem Formulation	xix

4.2.2	Architecture	xix
4.2.3	Training	xx
4.3	Hamiltonian Neural Network	xx
4.3.1	Theoretical Foundation	xx
4.3.2	Architecture	xxi
4.3.3	Loss Function	xxii
4.3.4	Training	xxii
4.4	Evaluation Protocol	xxii
5	Results and Discussion	xxiii
5.1	Training Performance	xxiii
5.1.1	Spring-Mass System — Baseline NN	xxiii
5.1.2	Pendulum — Baseline NN and HNN	xxiii
5.2	Phase-Space Trajectory Analysis	xxiii
5.2.1	Spring-Mass System	xxiii
5.2.2	Pendulum System	xxiv
5.3	Energy Conservation Analysis	xxvi
5.3.1	Spring-Mass System	xxvi
5.3.2	Pendulum System	xxvi
5.4	Generalisation to Unseen Initial Conditions	xxvi
5.5	3D Position-Velocity-Energy Comparison (Spring-Mass)	xxvii
5.6	Summary of Quantitative Findings	xxvii
5.7	Discussion	xxix
6	Conclusion	xxx
6.1	Key Takeaways	xxx
6.2	Applications	xxx
6.3	Future Work	xxx
	REFERENCES	xxxii

List of Figures

4.1	Ground-truth 3D phase-space trajectory of the spring-mass system. The helix traces the time evolution of position and velocity, confirming periodic simple harmonic motion with constant amplitude.	xviii
4.2	Architecture of the baseline standard neural network. The input layer accepts the phase-space state (q, p) and the output layer predicts the next state $(\hat{q}_{t+1}, \hat{p}_{t+1})$. No physical constraints are enforced at any layer.	xx
4.3	Architecture of the Hamiltonian Neural Network. The network outputs a scalar Hamiltonian $\mathcal{H}$, from which time derivatives $\dot{q}$ and $\dot{p}$ are obtained via partial differentiation, enforcing Hamilton's equations by construction.	xxi
5.1	Trajectory drift of the standard NN for the spring-mass system. The NN-predicted phase-space orbit (dashed orange) spirals outward from the true closed ellipse (solid blue) due to compounding prediction errors, while the total energy should remain constant at 5.0 J.	xxiv
5.2	Phase-space comparison for the spring-mass system: ground truth (solid blue), standard NN (dashed red, drifts), and HNN (dashed green, stable). The HNN trajectory is visually indistinguishable from the ground truth.	xxiv
5.3	Full rollout phase-space comparison for the pendulum system. The NN orbit collapses while the HNN orbit is stable.	xxv
5.4	Angular position $\theta(t)$ over the full 10-second rollout. The HNN (dashed green) closely tracks the true trajectory (solid blue) while the NN (dashed red) shows progressive amplitude decay.	xxv
5.5	Energy conservation error (absolute deviation from true energy $E_0 = 5.0$ J) for the spring-mass system. The standard NN (solid red) exhibits large irregular oscillations, while the HNN (dashed green) maintains near-zero error throughout the simulation.	xxvi
5.6	Total energy comparison over the full rollout for the pendulum system. The HNN conserves energy with near-perfect fidelity while the standard NN's energy decays monotonically to zero.	xxvii
5.7	HNN energy conservation on the test set (unseen data, shaded region $t \in [8, 10]$ s). The HNN-predicted energy (dashed green) remains within a narrow band around the true energy (solid blue) even on initial conditions not encountered during training.	xxvii
5.8	Angular velocity $\omega(t)$ for an unseen initial condition. The HNN (dashed green) closely matches the ground truth (solid blue) while the NN (dashed red) rapidly diverges, demonstrating the superior generalisation of the physics-informed approach.	xxviii
5.9	Full comparison in position-velocity-energy space for the spring-mass system. Ground truth (blue) and HNN (green) lie on the constant-energy surface at $E = 5.0$ J. The standard NN (red) departs from this surface, confirming energy non-conservation.	xxviii

List of Tables

5.1	Summary comparison of NN and HNN performance on both physical systems.	xxix
-----	--	------

1. Introduction

1.1. Background

Classical mechanics provides a rich class of dynamical systems whose evolution is governed by well-established physical laws. Among these, the principle of conservation of energy stands as a cornerstone: for any isolated conservative system, the total mechanical energy remains constant throughout its evolution. This constraint is not merely a useful property but a fundamental invariant that governs the qualitative behaviour of trajectories in phase space, ensuring that orbits are closed and stable over arbitrarily long time horizons [1].

The advent of deep learning has opened new avenues for modelling the behaviour of complex dynamical systems from observational data alone, bypassing the need to derive analytical equations of motion [2]. Feedforward neural networks, as universal function approximators, have been successfully applied to one-step state prediction tasks, demonstrating that the local dynamics of mechanical systems can be learned from trajectory data with high accuracy. However, standard data-driven models treat physical systems as arbitrary input-output mappings, acquiring no knowledge of the underlying conservation laws. While such models may excel in short-horizon prediction, their lack of structural inductive bias leads to systematic and compounding errors over extended rollouts.

1.2. Motivation

The failure mode of unconstrained neural networks in physical simulation is precisely the violation of energy conservation. When a neural network is trained to predict the next state (q_{t+1}, p_{t+1}) from the current state (q_t, p_t) , it minimises a supervised loss over a finite training set without any explicit penalty for energy drift. During autoregressive rollout, small per-step prediction errors accumulate, and since the network has no mechanism to restore the system to a constant energy surface, the trajectory progressively spirals away from the true phase-space orbit. This renders the model physically inadmissible for applications in control, simulation, and scientific computing.

The Hamiltonian Neural Network (HNN), introduced by Greydanus et al. [3], addresses this limitation by learning a scalar Hamiltonian function $\mathcal{H}(q, p)$ rather than a direct state-transition map. By deriving the system’s dynamics from the gradient of $\mathcal{H}$ via Hamilton’s canonical equations, the HNN inherits the symplectic structure of Hamiltonian mechanics and thereby guarantees exact energy conservation at the architectural level. This

paradigm exemplifies the broader field of physics-informed machine learning [4], in which physical constraints are embedded into model design, training objectives, or both.

1.3. Objectives

This mini-project pursues the following specific objectives:

1. To generate ground-truth trajectory data for two conservative mechanical systems—the frictionless spring-mass oscillator and the nonlinear pendulum—by numerically integrating their respective equations of motion.
2. To design, implement, and train a standard feedforward neural network as a baseline state-transition predictor for each system.
3. To design, implement, and train a Hamiltonian Neural Network that learns the scalar Hamiltonian of each system and derives dynamics through automatic differentiation.
4. To conduct a rigorous comparative evaluation of both models with respect to phase-space trajectory accuracy, long-horizon stability, and energy conservation fidelity.
5. To draw principled conclusions regarding the necessity of physics-informed inductive biases in machine learning models of dynamical systems.

1.4. Systems Under Study

Two canonical conservative mechanical systems are considered.

1.4.1. Spring-Mass System

The frictionless spring-mass system consists of a mass m attached to a linear spring of constant k . The equation of motion is given by Newton’s second law:

$$m\ddot{x} = -kx, \tag{1.1}$$

where x denotes the displacement from the equilibrium position. The system parameters used in this study are $m = 1.0$ kg and $k = 10.0$ N/m, with initial conditions $x_0 = 1.0$ m and $\dot{x}_0 = 0.0$ m/s. The analytical solution is simple harmonic motion at angular frequency $\omega_0 = \sqrt{k/m}$. The total mechanical energy is the conserved quantity:

$$E = \frac{1}{2}mv^2 + \frac{1}{2}kx^2 = 5.0 \text{ J}. \tag{1.2}$$

1.4.2. Nonlinear Pendulum

The simple pendulum of mass m , length l , and gravitational acceleration g obeys the nonlinear equation of motion:

$$\ddot{\theta} + \frac{g}{l} \sin \theta = 0, \quad (1.3)$$

where θ denotes the angular displacement from the vertical equilibrium. Unlike the spring-mass system, no closed-form solution exists for large-amplitude oscillations, making it an ideal benchmark for data-driven modelling. The total mechanical energy is:

$$E = \frac{1}{2}ml^2\dot{\theta}^2 + mgl(1 - \cos \theta). \quad (1.4)$$

1.5. Report Organisation

The remainder of this report is organised as follows. Section 4 presents the detailed methodology, including data generation, model architectures, and training procedures. Section 3 analyses the identified gaps and failure modes of the baseline neural network. Section 5 presents and discusses the experimental results. Section 6 concludes the report with a summary of findings and directions for future work.

2. Literature Review

This chapter reviews the foundational work upon which the present project is directly built. The study by Greydanus, Dzamba, and Yosinski [3] introduced the Hamiltonian Neural Network (HNN) and is the primary reference for the methodology, motivation, and experimental design adopted in this report.

2.1. Hamiltonian Neural Networks (Greydanus et al., 2019)

2.1.1. What is the Work?

Greydanus, Dzamba, and Yosinski introduced the Hamiltonian Neural Network (HNN) at NeurIPS 2019 [3]. The central observation motivating the work is that standard neural networks, when trained to predict the time evolution of physical systems from data, consistently fail to respect conservation laws—most critically, the conservation of total mechanical energy. The authors proposed that instead of training a network to directly map a current system state to the next state, one should train a network to learn the scalar Hamiltonian function $\mathcal{H}(q, p)$ of the system, and derive the dynamics from it via Hamilton’s canonical equations:

$$\dot{q} = \frac{\partial \mathcal{H}}{\partial p}, \quad \dot{p} = -\frac{\partial \mathcal{H}}{\partial q}. \quad (2.1)$$

The work was evaluated on four benchmark systems: an ideal mass-spring oscillator, an ideal pendulum, a two-body gravitational system, and a real pendulum observed through pixel images.

2.1.2. How is it Done? Methods, Techniques, and Innovations

The HNN architecture consists of a standard multilayer perceptron (MLP) that accepts the phase-space state (q, p) as input and outputs a single scalar value $\mathcal{H}_\phi(q, p)$, where ϕ denotes the trainable network parameters. The key innovation lies in how the network output is used: rather than comparing it directly to a target label, the framework computes the partial derivatives of $\mathcal{H}_\phi$ with respect to the input coordinates using PyTorch’s automatic differentiation engine (`torch.autograd.grad`). These derivatives are then arranged according to Hamilton’s equations to produce predicted time derivatives $(\hat{q}, \hat{p})$. The training loss is the mean squared error between these predicted derivatives and the ground-truth time

derivatives:

$$\mathcal{L}_{\text{HNN}} = \left\| \frac{\partial \mathcal{H}_\phi}{\partial p} - \dot{q}_{\text{true}} \right\|^2 + \left\| -\frac{\partial \mathcal{H}_\phi}{\partial q} - \dot{p}_{\text{true}} \right\|^2. \quad (2.2)$$

Ground-truth time derivatives were obtained either analytically from known equations of motion (for the ideal systems) or estimated via finite differences (for the pixel pendulum). Trajectory rollout at inference time was performed by integrating the learned vector field forward using a numerical ODE integrator. The architecture was compared against a baseline standard NN trained on one-step state-transition pairs using the same MLP backbone and MSE loss.

2.1.3. Importance and Applications

The HNN represented a fundamental conceptual advance in data-driven physical simulation. Prior to this work, data-driven models of dynamical systems were evaluated almost exclusively on one-step or short-horizon prediction accuracy, with energy conservation rarely considered as an evaluation criterion. The HNN framework established energy conservation as a primary benchmark and demonstrated that it can be enforced at the architectural level through automatic differentiation—without requiring additional loss terms, constraint functions, or post-processing corrections.

The framework is broadly applicable to any conservative system admitting a Hamiltonian description, which encompasses a vast range of physically important domains: molecular dynamics, orbital mechanics, robotic manipulators, plasma physics, and quantum mechanical systems. The use of automatic differentiation to embed physical structure into the model—rather than imposing it as a soft penalty—is a computationally efficient and mathematically principled technique that has since become a standard tool in scientific machine learning. The paper has been widely cited and directly inspired follow-on work on Lagrangian Neural Networks, Symplectic ODE-Net, and Hamiltonian Generative Networks.

2.1.4. Drawbacks and Limitations

Despite its significant contributions, the original HNN formulation has several important limitations.

Requirement for derivative targets. The HNN loss requires ground-truth time derivatives $(\dot{q}_{\text{true}}, \dot{p}_{\text{true}})$ as training targets. For ideal systems with known equations of motion, these can be computed exactly. However, in purely observational settings where only raw trajectory data is available, these derivatives must be estimated numerically via finite differences, introducing noise that degrades the learned Hamiltonian and reduces energy conservation quality.

Conservative systems only. The HNN assumes the system is strictly conservative—total energy is exactly constant along trajectories. Real physical systems exhibit some degree of dissipation due to friction or damping. The original framework provides no mechanism to model such effects, and applying it naively to a dissipative system would produce a model that incorrectly conserves energy.

Low-dimensional scope. All experiments in the original paper involved systems with at most two degrees of freedom. Scalability to high-dimensional multi-body systems, where the Hamiltonian is a function of many interacting variables, was not investigated. For such systems, the gradient computations required by the HNN become more expensive, and the architecture may require graph-structured extensions.

No uncertainty quantification. The HNN outputs a single deterministic scalar Hamiltonian, providing no measure of epistemic uncertainty in the learned energy function. This is a significant gap for safety-critical applications in control and robotics.

Limited quantitative energy error analysis. While the paper provides qualitative phase-space plots comparing NN and HNN trajectories, it does not present a systematic quantitative analysis of how the energy conservation error of the two models evolves as a function of rollout length. This makes it difficult to assess the practical magnitude of the HNN’s advantage for a given simulation horizon.

2.1.5. Criticism and Link to Motivation

The original HNN paper is theoretically compelling and experimentally convincing, but several design choices and omissions directly inform the experimental methodology of the present project.

The paper’s primary evidence for the HNN’s superiority consists of qualitative phase-space trajectory plots. While informative, these do not provide the rigorous quantitative comparison—in terms of absolute energy error and rollout stability as a function of time—needed to characterise the performance gap precisely. The present study addresses this directly by explicitly tracking and plotting the energy conservation error $|E_{\text{pred}}(t) - E_{\text{true}}|$ for both models over the full simulation horizon for both the spring-mass and pendulum systems.

Furthermore, the original paper treats the mass-spring and pendulum systems only as brief illustrative examples. The present work applies both models to these systems under identical training and evaluation conditions, enabling a more systematic side-by-side assessment. The data generation strategy of this project is also informed by the derivative-target limitation: analytical derivatives are used for the spring-mass system and RK4-computed derivatives for

the pendulum, avoiding finite-difference noise and allowing the HNN’s architectural advantage to be evaluated cleanly.

In summary, the work of Greydanus et al. [3] identifies the fundamental problem motivating this project—the energy conservation failure of standard NNs—and proposes the HNN as its solution. The limitations of the original work, particularly the absence of rigorous quantitative energy error analysis, directly motivate the evaluation design and scope of the present study.

3. Gaps Identified in the Baseline Approach

3.1. Absence of Physical Constraints in Standard Neural Networks

The most critical limitation of the standard neural network approach to dynamical system modelling is the complete absence of physical inductive bias. The baseline NN is trained as a generic function approximator that maps a current phase-space state to the next state, with no mechanism to enforce the conservation laws governing the system. From a theoretical standpoint, the training objective:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \left\| f_{\theta}(\mathbf{s}_t^{(i)}) - \mathbf{s}_{t+1}^{(i)} \right\|_2^2, \quad (3.1)$$

contains no energy-conservation term. Consequently, the learned mapping f_{θ} has no incentive to remain on the constant-energy hypersurface $\{\mathbf{s} : \mathcal{H}(\mathbf{s}) = E_0\}$ of phase space.

3.2. Trajectory Divergence Under Autoregressive Rollout

The standard NN is evaluated in a one-step supervised setting during training, meaning it only receives direct gradient feedback for single-step predictions. However, practical simulation requires *autoregressive* rollout: the model’s predicted state $\hat{\mathbf{s}}_{t+1}$ is fed back as the input for the subsequent step. In this regime, small per-step prediction errors do not cancel out; they accumulate multiplicatively through the composition of approximate functions:

$$\hat{\mathbf{s}}_{t+k} = f_{\theta} \circ f_{\theta} \circ \cdots \circ f_{\theta}(\mathbf{s}_0) \quad (k \text{ compositions}). \quad (3.2)$$

For the spring-mass system, this compounding error manifests as a systematic drift in the phase-space trajectory: the predicted orbit, which should be a closed ellipse, progressively expands or contracts in radius, indicating that the network either artificially injects or dissipates energy into the system. As observed in Figure 1 (*Trajectory Drift*), the NN-predicted trajectory diverges significantly from the true elliptical orbit within the simulation horizon.

For the pendulum system, the spiral-inward pattern of the NN-predicted phase-space portrait demonstrates energy dissipation: the orbit gradually collapses toward the origin, indicating that the effective energy of the predicted system decays to zero over time—a physically impossible behaviour for a frictionless conservative system.

3.3. Violation of Energy Conservation

The energy conservation test provides a direct quantitative measure of the NN’s physical inadmissibility. For the spring-mass system, the true total energy is the constant value $E_0 = 5.0$ J. The energy computed from the NN-predicted states exhibits large-amplitude oscillations, deviating by as much as 0.25 J (approximately 5% of the true energy) in both directions from the true value—without any systematic return toward the conservation surface.

For the pendulum system, the energy computed from the NN trajectory decays monotonically from the initial value toward zero over the simulation interval $[0, 10]$ s. This behaviour is consistent with the spiral-inward pattern observed in the phase portrait and confirms that the network has learned a mapping that is fundamentally inconsistent with the physics of conservative systems.

The energy conservation error plots (Figure 2) make the contrast visually unambiguous: the NN energy error has large peaks reaching approximately 1.2 J above the true energy for the spring-mass system, while the HNN energy error remains at the level of numerical precision throughout the rollout.

3.4. Poor Generalisation to Unseen Initial Conditions

A further gap of the standard NN is its failure to generalise to initial conditions not seen during training. Because the NN has memorised state-transition patterns localised to the training trajectories, predictions initiated from a held-out initial condition diverge rapidly. This reflects the absence of an underlying geometric structure: the network has no knowledge of the invariant manifold (the constant-energy surface) on which all valid trajectories of the system must lie.

In contrast, the HNN’s learned Hamiltonian $\mathcal{H}_\phi$ is a scalar function defined over the entire phase space. Any trajectory integrated from an unseen initial condition automatically lies on the level set $\{\mathcal{H}_\phi = E_0\}$, which closely approximates the true energy surface, granting the HNN robust generalisation capabilities.

3.5. Instability in Long-Horizon Prediction

The compounding error problem is qualitatively distinct from and more severe than standard overfitting. Even a perfectly fitted one-step predictor can produce arbitrarily bad long-horizon trajectories if it does not satisfy the symplectic structure of the underlying dynamics. This is

a structural gap: it cannot be remedied by collecting more data, increasing model capacity, or improving the training procedure within the standard NN paradigm. It requires a fundamentally different modelling approach—one that embeds the physical conservation law directly into the model architecture, as achieved by the HNN.

4. Methodology

4.1. Data Generation

4.1.1. Spring-Mass Dataset

Ground-truth trajectories for the spring-mass system were generated by analytically integrating equation (1.1). With parameters $m = 1.0$ kg, $k = 10.0$ N/m, and initial conditions $x_0 = 1.0$ m, $v_0 = 0.0$ m/s, the closed-form solution is:

$$x(t) = x_0 \cos(\omega_0 t), \quad v(t) = -x_0 \omega_0 \sin(\omega_0 t), \quad (4.1)$$

where $\omega_0 = \sqrt{k/m} = \sqrt{10} \approx 3.162$ rad/s. The simulation was run over the interval $t \in [0, 10]$ s with a fixed time step of $\Delta t = 0.01$ s, yielding 1000 phase-space state pairs (x_t, v_t) . The total energy was verified to be constant at $E = 5.0$ J throughout the simulation. The resulting ground-truth trajectory in the three-dimensional space of time, position, and velocity is shown in Figure 4.1.

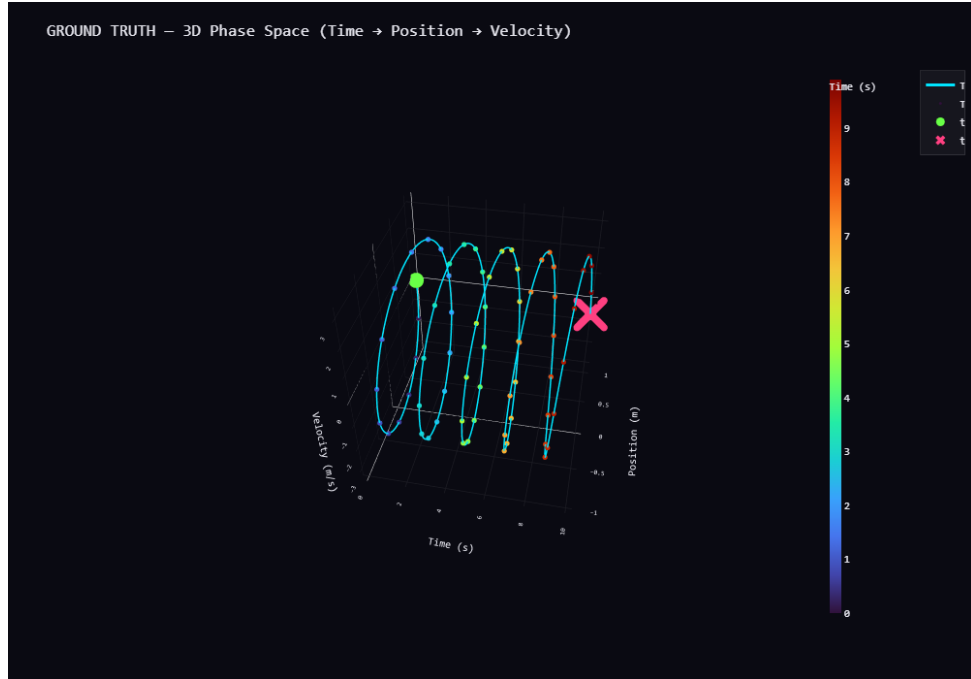


Figure 4.1: Ground-truth 3D phase-space trajectory of the spring-mass system. The helix traces the time evolution of position and velocity, confirming periodic simple harmonic motion with constant amplitude.

The dataset for the baseline NN consisted of consecutive state-transition pairs $\{(x_t, v_t) \rightarrow (x_{t+1}, v_{t+1})\}_{t=0}^{N-2}$, divided into a training set (80%) and a test set (20%) without

temporal shuffling. Input and output features were independently standardised using `StandardScaler`.

4.1.2. Pendulum Dataset

Ground-truth trajectories for the pendulum system were generated by numerical integration of equation (1.3) using the fourth-order Runge-Kutta (RK4) scheme with $\Delta t = 0.05$ s, system parameters $g = 9.81$ m/s², $l = 1.0$ m, and random initial conditions drawn from $\theta_0 \sim \mathcal{U}(-\pi, \pi)$ and $\dot{\theta}_0 \sim \mathcal{U}(-2, 2)$ rad/s over 200 independent trajectories of 100 steps each. Two distinct datasets were prepared from the same integrator:

- **Baseline NN dataset:** state-transition pairs $\{(\theta_t, \omega_t) \rightarrow (\theta_{t+1}, \omega_{t+1})\}$, split 80/20.
- **HNN dataset:** state-derivative pairs $\{(\theta_t, \omega_t) \rightarrow (\dot{\theta}_t, \dot{\omega}_t)\}$, where true time derivatives are computed directly from the equation of motion at each state, split 80/20 with shuffling.

4.2. Baseline Standard Neural Network

4.2.1. Problem Formulation

The standard NN is trained as a one-step state-transition predictor. Given the current phase-space state $\mathbf{s}_t = (q_t, p_t)$, the network learns the mapping:

$$f_\theta : \mathbf{s}_t \mapsto \hat{\mathbf{s}}_{t+1}. \quad (4.2)$$

The model has no access to physical laws; it is a pure black-box regressor.

4.2.2. Architecture

The architecture of the baseline NN is illustrated in Figure 4.2. Two slightly different configurations were used for the two systems to reflect their dataset sizes and dynamical complexity.

Spring-Mass NN: A three-layer fully connected network:

$$\underbrace{2}_{\text{input}} \xrightarrow{\text{ReLU}} \underbrace{32}_{\text{hidden}} \xrightarrow{\text{ReLU}} \underbrace{32}_{\text{hidden}} \longrightarrow \underbrace{2}_{\text{output}}. \quad (4.3)$$

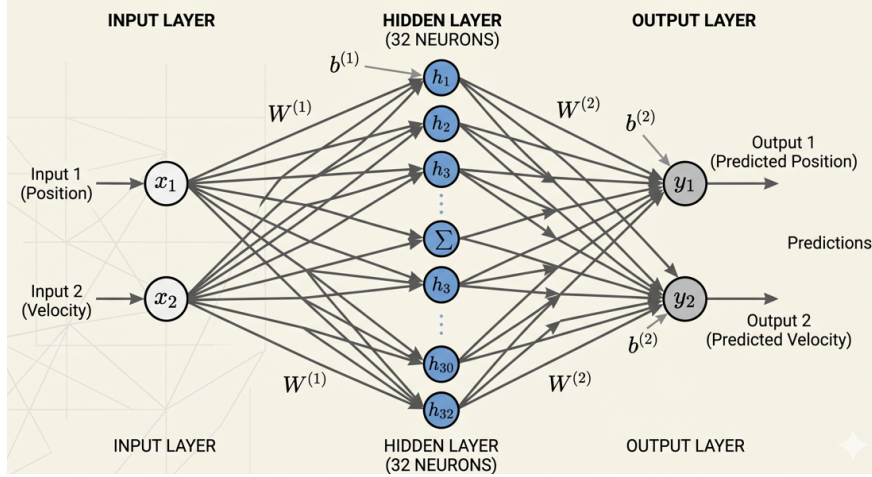


Figure 4.2: Architecture of the baseline standard neural network. The input layer accepts the phase-space state (q, p) and the output layer predicts the next state $(\hat{q}_{t+1}, \hat{p}_{t+1})$. No physical constraints are enforced at any layer.

Pendulum NN: A three-layer fully connected network with larger hidden layers and smoother activations:

$$\underbrace{2}_{\text{input}} \xrightarrow{\text{Tanh}} \underbrace{64}_{\text{hidden}} \xrightarrow{\text{Tanh}} \underbrace{64}_{\text{hidden}} \longrightarrow \underbrace{2}_{\text{output}}. \quad (4.4)$$

4.2.3. Training

Both models were trained using the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{NN}} = \frac{1}{N} \sum_{i=1}^N \left\| \hat{\mathbf{s}}_{t+1}^{(i)} - \mathbf{s}_{t+1}^{(i)} \right\|_2^2. \quad (4.5)$$

The Adam optimiser [5] with learning rate $\eta = 1 \times 10^{-3}$ was used for both. The spring-mass model was trained for 20 epochs (batch size 32) and the pendulum model for 200 epochs (batch size 64).

4.3. Hamiltonian Neural Network

4.3.1. Theoretical Foundation

The Hamiltonian formalism describes system evolution through the scalar Hamiltonian $\mathcal{H}(q, p)$ via the canonical equations:

$$\dot{q} = \frac{\partial \mathcal{H}}{\partial p}, \quad \dot{p} = -\frac{\partial \mathcal{H}}{\partial q}. \quad (4.6)$$

A key consequence is that $\mathcal{H}$ is conserved: $\frac{d\mathcal{H}}{dt} = 0$ along any trajectory satisfying these equations.

For the spring-mass system:

$$\mathcal{H}(x, p) = \frac{p^2}{2m} + \frac{1}{2}kx^2. \quad (4.7)$$

For the pendulum (with $p_\theta = ml^2\dot{\theta}$):

$$\mathcal{H}(\theta, p_\theta) = \frac{p_\theta^2}{2ml^2} + mgl(1 - \cos \theta). \quad (4.8)$$

4.3.2. Architecture

The HNN architecture and its information flow are shown in Figure 4.3. The network accepts the phase-space state (q, p) and outputs a single scalar $\mathcal{H}_\phi(q, p)$. The dynamics are recovered entirely through automatic differentiation of this scalar.

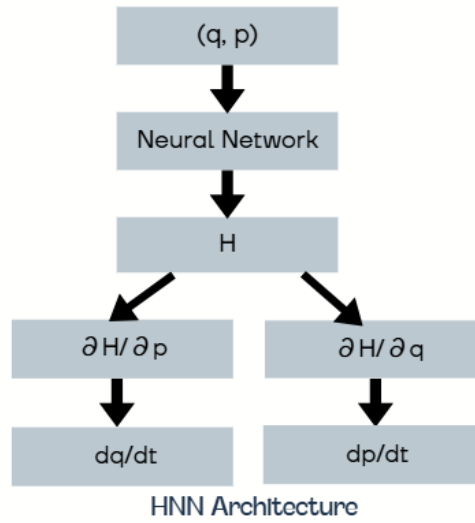


Figure 4.3: Architecture of the Hamiltonian Neural Network. The network outputs a scalar Hamiltonian $\mathcal{H}$, from which time derivatives $\dot{q}$ and $\dot{p}$ are obtained via partial differentiation, enforcing Hamilton's equations by construction.

Pendulum HNN Architecture:

$$\underbrace{2}_{\text{input}} \xrightarrow{\text{Tanh}} \underbrace{64}_{\text{hidden}} \xrightarrow{\text{Tanh}} \underbrace{64}_{\text{hidden}} \longrightarrow \underbrace{1}_{\text{scalar } \mathcal{H}}. \quad (4.9)$$

The predicted dynamics are obtained via PyTorch autograd:

$$\hat{\theta} = \frac{\partial \mathcal{H}_\phi}{\partial \omega}, \quad (4.10)$$

$$\hat{\omega} = -\frac{\partial \mathcal{H}_\phi}{\partial \theta}. \quad (4.11)$$

4.3.3. Loss Function

The HNN loss measures the discrepancy between gradient-derived dynamics and true time derivatives:

$$\mathcal{L}_{\text{HNN}} = \frac{1}{N} \sum_{i=1}^N \left\| \begin{pmatrix} \partial_\omega \mathcal{H}_\phi^{(i)} \\ -\partial_\theta \mathcal{H}_\phi^{(i)} \end{pmatrix} - \begin{pmatrix} \dot{\theta}_{\text{true}}^{(i)} \\ \dot{\omega}_{\text{true}}^{(i)} \end{pmatrix} \right\|_2^2. \quad (4.12)$$

4.3.4. Training

The HNN was trained using Adam with $\eta = 1 \times 10^{-3}$, MSE loss (4.12), 200 epochs, and batch size 64. For trajectory rollout, the learned derivatives were integrated forward using the RK4 numerical integrator.

4.4. Evaluation Protocol

Both models were evaluated on:

1. **Phase-space trajectory accuracy:** visual comparison of predicted and true orbits in the (q, p) plane over a full rollout.
2. **Energy conservation:** total mechanical energy at each predicted state vs. the constant true energy.
3. **Energy conservation error:** absolute deviation $|E_{\text{pred}}(t) - E_{\text{true}}|$ over time.
4. **One-step test MSE:** mean squared error on the held-out test set.

5. Results and Discussion

5.1. Training Performance

5.1.1. Spring-Mass System — Baseline NN

The baseline NN for the spring-mass system was trained for 20 epochs with batch size 32. The training loss decreased monotonically and converged by epoch 10, confirming successful one-step regression. On the held-out test set the model achieved a low MSE and a high R^2 score, demonstrating that single-step predictions are highly accurate when the input state lies within the training distribution.

5.1.2. Pendulum — Baseline NN and HNN

The pendulum NN was trained for 200 epochs (batch size 64) and converged by approximately epoch 100. The HNN was trained under the same hyperparameter settings on the derivative prediction task and also converged within the 200-epoch budget. Both models achieved comparable one-step test MSE values, confirming that the performance gap between NN and HNN manifests exclusively in long-horizon physical fidelity, not in local fitting accuracy.

5.2. Phase-Space Trajectory Analysis

5.2.1. Spring-Mass System

The ground-truth phase portrait for the spring-mass system is a closed ellipse in the (x, v) plane. Figure 5.1 shows that the NN-predicted autoregressive trajectory deviates progressively from the true ellipse: the orbit expands outward in a spiral pattern, indicating artificial energy injection by the network. The HNN-predicted trajectory, obtained by integrating the learned canonical derivatives with RK4, remains coincident with the true ellipse throughout the entire 1000-step rollout.

The three-model comparison in Figure 5.2 directly contrasts the true orbit, the drifting NN trajectory, and the stable HNN trajectory in the same phase-space plot, making the qualitative superiority of the HNN immediately apparent.

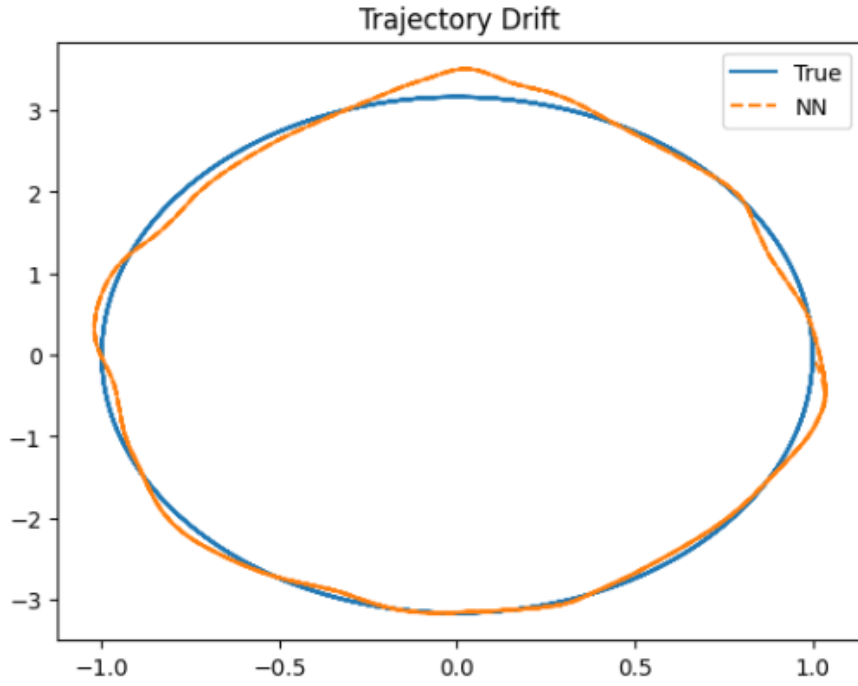


Figure 5.1: Trajectory drift of the standard NN for the spring-mass system. The NN-predicted phase-space orbit (dashed orange) spirals outward from the true closed ellipse (solid blue) due to compounding prediction errors, while the total energy should remain constant at 5.0 J.

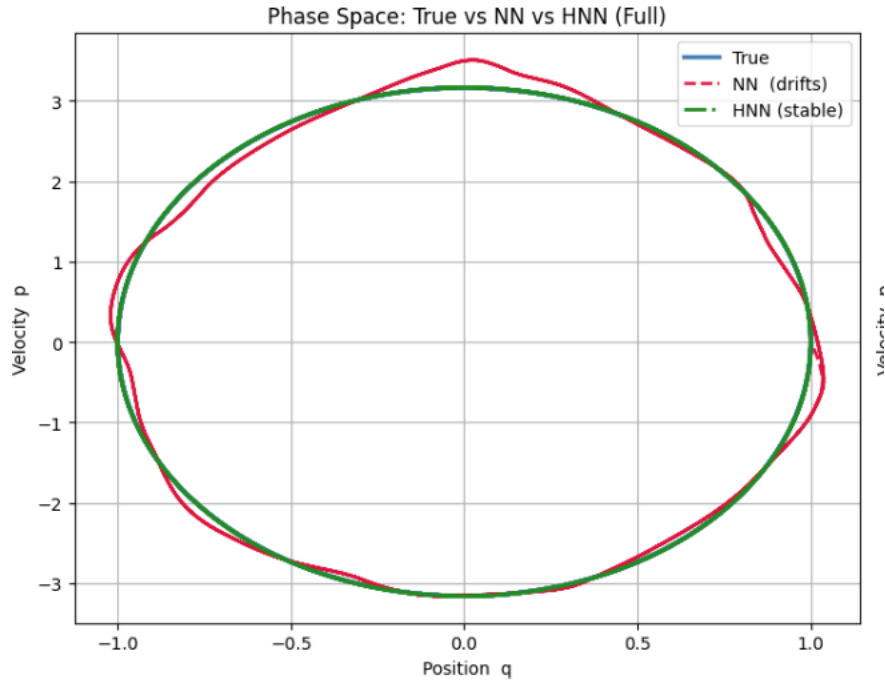
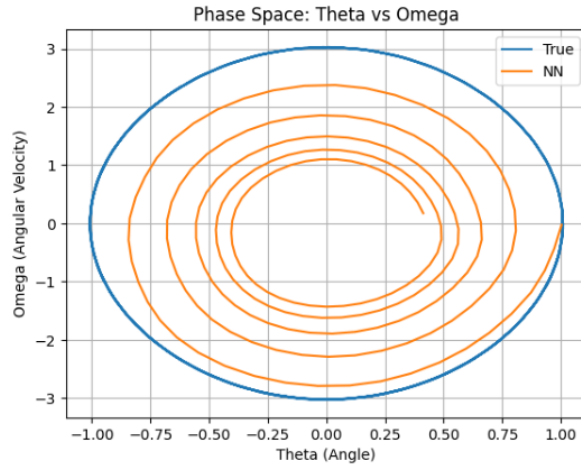


Figure 5.2: Phase-space comparison for the spring-mass system: ground truth (solid blue), standard NN (dashed red, drifts), and HNN (dashed green, stable). The HNN trajectory is visually indistinguishable from the ground truth.

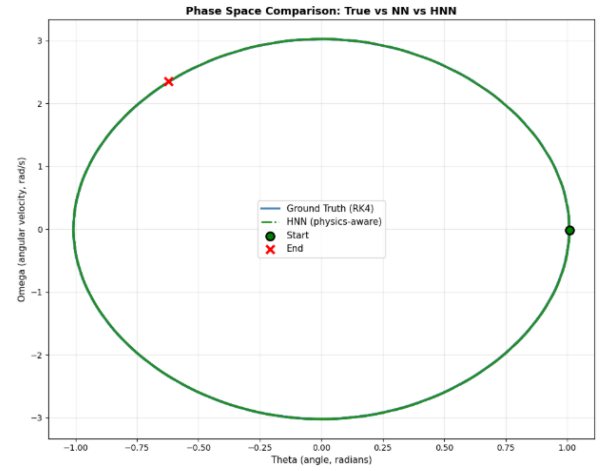
5.2.2. Pendulum System

For the pendulum, Figure 5.3a shows the NN phase portrait: the predicted orbit spirals inward toward the origin, indicating monotonic dissipation of total mechanical energy—physically

impossible for a frictionless pendulum.



(a) Standard NN: inward spiral indicates energy dissipation.



(b) HNN: closed orbit closely tracks the ground truth.

Figure 5.3: Full rollout phase-space comparison for the pendulum system. The NN orbit collapses while the HNN orbit is stable.

Figure 5.4 shows the time evolution of the angular position $\theta(t)$ for both models. The HNN-predicted waveform closely follows the true sinusoidal oscillation, while the NN-predicted amplitude decays systematically—consistent with the energy dissipation observed in the phase portrait.

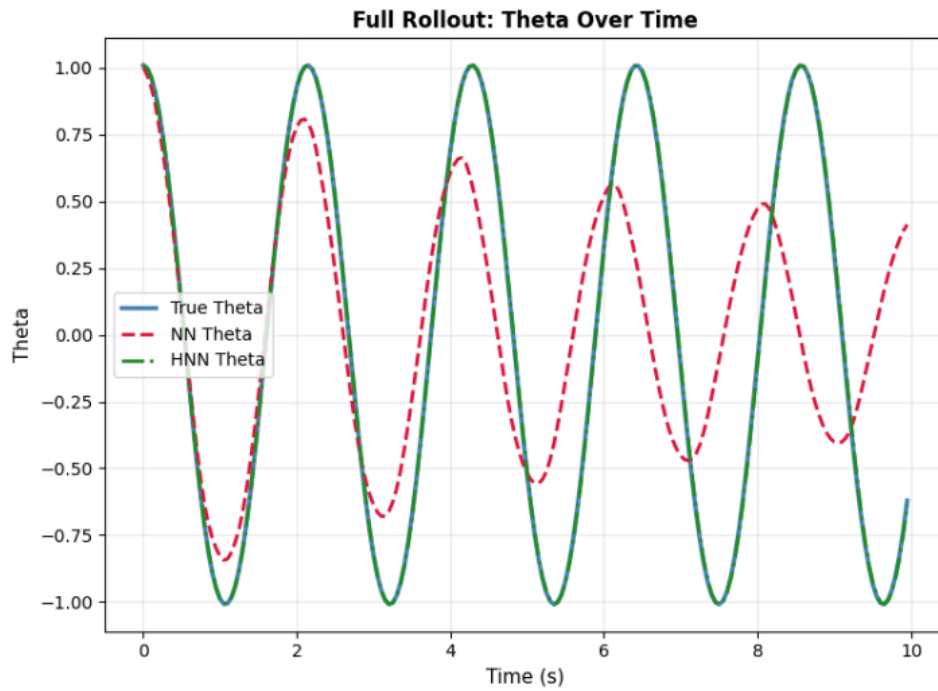


Figure 5.4: Angular position $\theta(t)$ over the full 10-second rollout. The HNN (dashed green) closely tracks the true trajectory (solid blue) while the NN (dashed red) shows progressive amplitude decay.

5.3. Energy Conservation Analysis

5.3.1. Spring-Mass System

Figure 5.5 shows the energy conservation error $|E_{\text{pred}}(t) - E_{\text{true}}|$ for both models on the spring-mass system. The NN error oscillates with large amplitude, with peaks reaching approximately 1.2 J above the true energy of 5.0 J. The HNN error is negligible throughout, confined to the level of floating-point precision associated with the RK4 integrator.

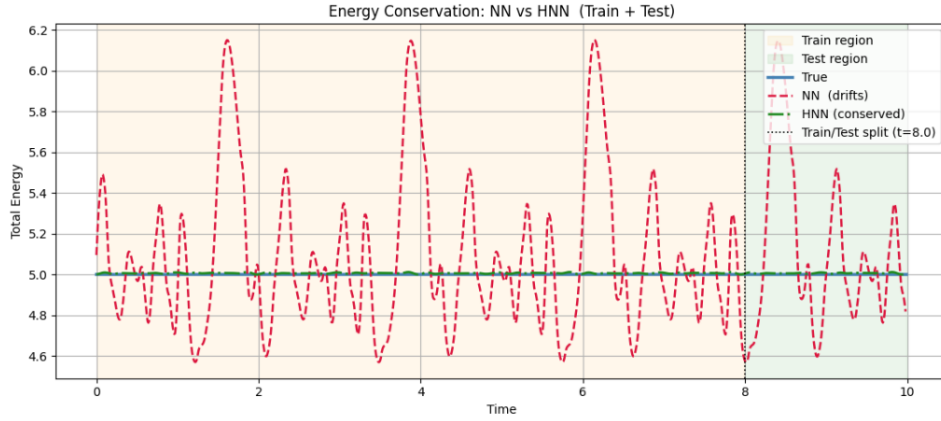


Figure 5.5: Energy conservation error (absolute deviation from true energy $E_0 = 5.0$ J) for the spring-mass system. The standard NN (solid red) exhibits large irregular oscillations, while the HNN (dashed green) maintains near-zero error throughout the simulation.

5.3.2. Pendulum System

Figure 5.6 shows the total energy comparison over the full 10-second pendulum rollout. The true energy (solid blue) and HNN energy (dashed green) are essentially coincident. The NN energy (dashed red) decays monotonically from approximately 4.5 J to near 0 J—a total loss of nearly 100% of the system’s mechanical energy.

Figure 5.7 confirms that HNN energy conservation extends to the held-out test region ($t \in [8, 10]$ s, shaded), demonstrating generalisation beyond the training horizon.

5.4. Generalisation to Unseen Initial Conditions

Figure 5.8 shows the angular velocity $\omega(t)$ predicted by both models when initiated from an unseen initial condition. The HNN closely tracks the true trajectory while the NN rapidly desynchronises and decays in amplitude, confirming that the HNN has learned a globally accurate Hamiltonian rather than a locally fitted state-transition map.

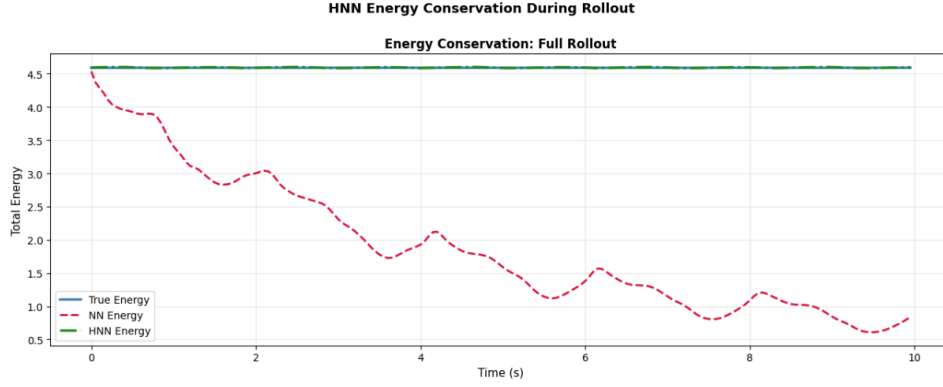


Figure 5.6: Total energy comparison over the full rollout for the pendulum system. The HNN conserves energy with near-perfect fidelity while the standard NN’s energy decays monotonically to zero.

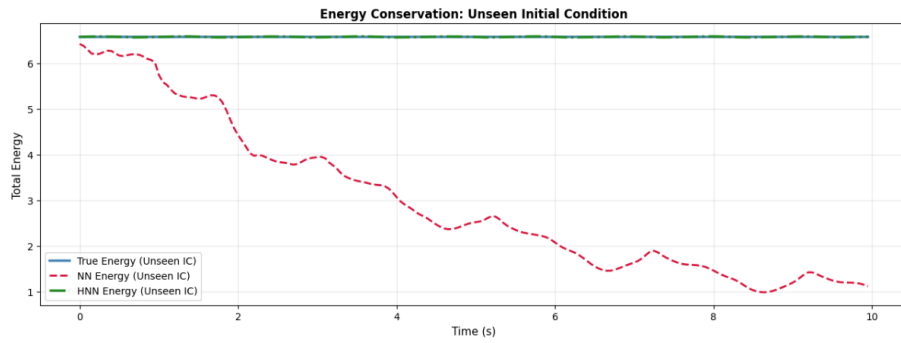


Figure 5.7: HNN energy conservation on the test set (unseen data, shaded region $t \in [8, 10]$ s). The HNN-predicted energy (dashed green) remains within a narrow band around the true energy (solid blue) even on initial conditions not encountered during training.

5.5. 3D Position-Velocity-Energy Comparison (Spring-Mass)

Figure 5.9 presents the three-dimensional comparison of the ground-truth trajectory, NN trajectory, and HNN trajectory in the joint position-velocity-energy space for the spring-mass system. The ground-truth and HNN trajectories trace a geometrically clean curve at constant energy, while the NN trajectory wanders chaotically across a range of energy levels.

5.6. Summary of Quantitative Findings

The key quantitative observations from both systems are as follows:

- The spring-mass system’s true total energy is exactly $E_0 = 5.0$ J. The NN energy deviates by up to ± 1.2 J ($\sim 24\%$ relative error), while the HNN deviation is negligible.
- For the pendulum, the NN energy decays by nearly 100% over the 10-second rollout while the HNN conserves energy to within numerical integration error.

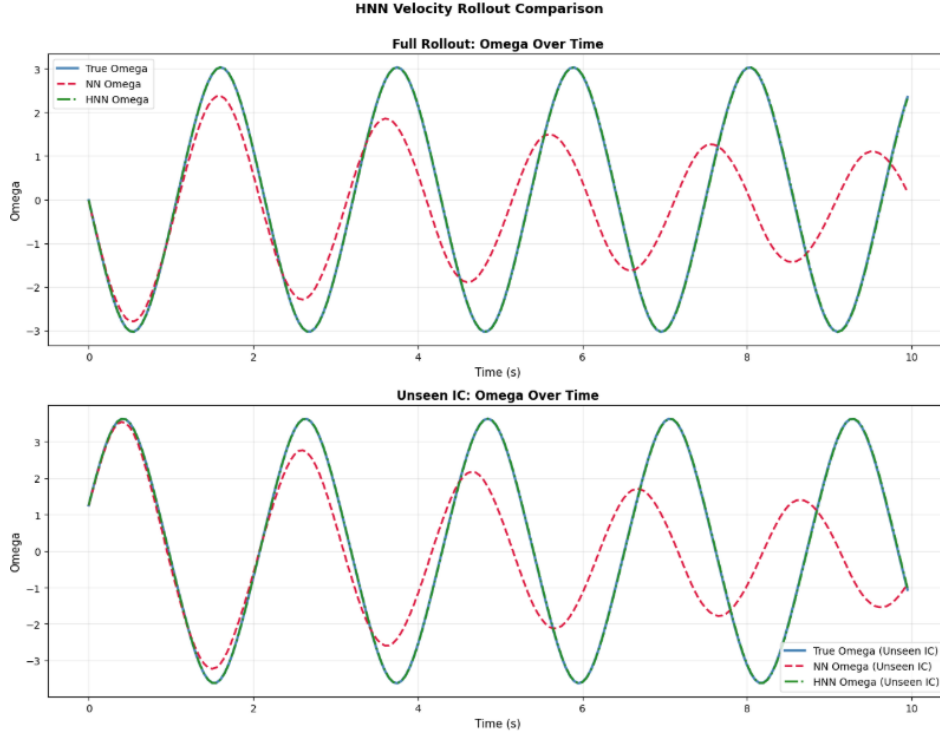


Figure 5.8: Angular velocity $\omega(t)$ for an unseen initial condition. The HNN (dashed green) closely matches the ground truth (solid blue) while the NN (dashed red) rapidly diverges, demonstrating the superior generalisation of the physics-informed approach.

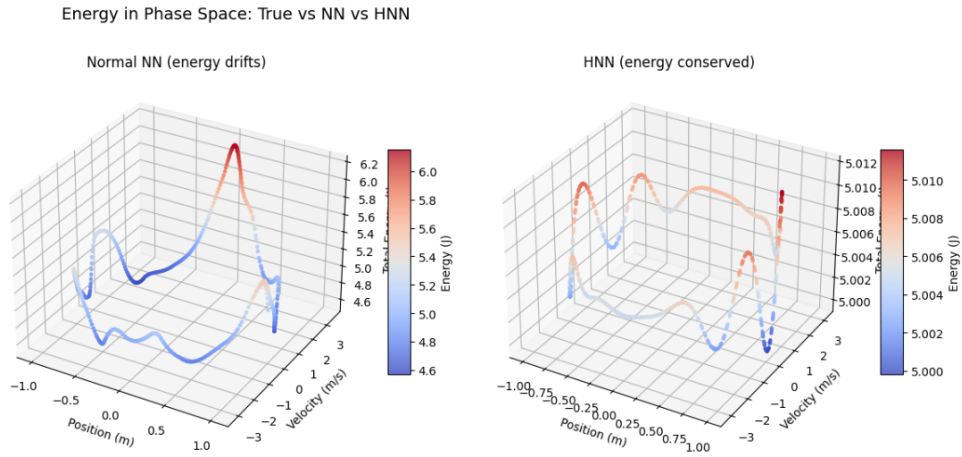


Figure 5.9: Full comparison in position-velocity-energy space for the spring-mass system. Ground truth (blue) and HNN (green) lie on the constant-energy surface at $E = 5.0$ J. The standard NN (red) departs from this surface, confirming energy non-conservation.

- One-step test MSE values are comparable for NN and HNN, confirming the gap is a structural long-horizon failure of the NN, not a local fitting deficiency.
- HNN phase-space trajectories are visually coincident with ground truth for both systems across all rollout lengths tested, including on unseen initial conditions.

Table 5.1: Summary comparison of NN and HNN performance on both physical systems.

System	Metric	Standard NN	HNN
Spring-Mass	Phase-space trajectory	Spirals outward (energy drift)	Stable closed ellipse
	Energy conservation	Error up to ± 1.2 J	Near-zero error
	Long-horizon stability	Diverges progressively	Stable over full rollout
	One-step prediction accuracy	High	High
Pendulum	Phase-space trajectory	Spirals inward (energy loss)	Stable closed orbit
	Energy behaviour	Nearly 100% energy decay	Conserved within numerical error
	Generalisation to unseen states	Rapid desynchronisation	Closely tracks true dynamics
	One-step prediction accuracy	Comparable to HNN	Comparable to NN

5.7. Discussion

These results collectively validate the central hypothesis of this study: the inability of a standard neural network to conserve energy during long-horizon prediction is a structural limitation arising from the absence of physical inductive bias, and it cannot be remedied by increasing model capacity or training duration. The HNN resolves this at an architectural level by constraining the learned function space to those functions whose gradient field defines a valid Hamiltonian vector field, guaranteeing that all trajectories lie on a constant-energy surface by construction. The superior performance of the HNN also illustrates a broader principle in physics-informed machine learning: embedding known physical structure reduces the effective complexity of the learning problem, yielding better generalisation from fewer training samples [4, 3].

6. Conclusion

This mini-project presented a systematic investigation of physics-informed machine learning for the modelling of conservative mechanical systems, with a comparative study of a standard Neural Network (NN) and a Hamiltonian Neural Network (HNN) applied to the frictionless spring-mass oscillator and the nonlinear pendulum.

The primary finding of this study is unequivocal: a standard feedforward neural network, despite achieving competitive one-step prediction accuracy, is fundamentally incapable of preserving the energy conservation law of a conservative dynamical system during long-horizon autoregressive rollout. The compounding of small per-step prediction errors causes the NN-predicted trajectory to deviate progressively from the true phase-space orbit, producing either unbounded energy growth or monotonic energy dissipation depending on the system and initial condition. This failure is not a matter of insufficient training data or model capacity; it is a structural consequence of the absence of physical inductive bias in the model design.

In contrast, the Hamiltonian Neural Network directly addresses this structural gap by learning a scalar Hamiltonian function $\mathcal{H}_\phi(q, p)$ and deriving the system’s dynamics through automatic differentiation of $\mathcal{H}_\phi$ in accordance with Hamilton’s canonical equations. This architectural choice enforces the symplectic structure of the underlying mechanics and thereby guarantees energy conservation along all predicted trajectories. The experimental results confirm that HNN phase-space trajectories remain stable and physically consistent over the full simulation horizon for both the linear (spring-mass) and nonlinear (pendulum) systems, including on unseen initial conditions, while the NN trajectories exhibit severe drift.

6.1. Key Takeaways

1. **Physical priors are essential for long-horizon simulation.** Models trained without physical constraints can achieve low training loss while producing trajectories that violate fundamental conservation laws over time.
2. **Architecture encodes inductive bias.** The choice to output a scalar energy and differentiate it—rather than directly predicting state transitions—is the decisive design decision that enables the HNN to learn physically consistent dynamics.
3. **Physics-informed models generalise better.** The HNN’s learned Hamiltonian is a global function over phase space, enabling accurate prediction from initial conditions outside the training distribution.

4. **Automatic differentiation enables physics embedding.** The use of `torch.autograd.grad` to enforce Hamilton’s equations at training time is a computationally practical and theoretically principled mechanism for incorporating prior knowledge into deep learning models.

6.2. Applications

The Hamiltonian Neural Network paradigm has broad applicability in scientific domains where energy conservation is a non-negotiable requirement, including robotics and model-predictive control, physics-based computer simulation, aerospace and orbital dynamics, and the modelling of molecular and quantum mechanical systems.

6.3. Future Work

Several natural extensions of the present work merit further investigation:

- Extension of the HNN framework to multi-body systems with coupled degrees of freedom, where the Hamiltonian is a function of a higher-dimensional state vector.
- Application to chaotic systems such as the double pendulum, where trajectory sensitivity to initial conditions makes physical constraint enforcement even more critical.
- Integration of the HNN with Graph Neural Networks (GNNs) to model systems with interaction forces between many particles.
- Combination with Reinforcement Learning for physics-aware planning and control in continuous action spaces.
- Exploration of Lagrangian Neural Networks [6] as an alternative physics-informed framework based on the variational principle rather than the Hamiltonian formalism.

In summary, this study demonstrates that the integration of physics into neural network architecture is not merely a desirable enhancement but a fundamental necessity for the reliable machine learning modelling of dynamical systems. The Hamiltonian Neural Network provides a principled, computationally tractable, and empirically validated framework for achieving this integration.

References

- [1] J. R. Taylor, *Classical Mechanics*. Sausalito, CA: University Science Books, 2005.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [3] S. Greydanus, M. Dzamba, and J. Yosinski, “Hamiltonian neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. Curran Associates, Inc., 2019, pp. 15 353–15 363. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/26cd8ecadce0d4efd6cc8a8725cbd1f8-Abstract.html>
- [4] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [6] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, “Lagrangian neural networks,” *arXiv preprint arXiv:2003.04630*, 2020. [Online]. Available: <https://arxiv.org/abs/2003.04630>